ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені ІВАНА ФРАНКА

Факультет прикладної математики та інформатики

Бази даних та інформаційні системи

ЛАБОРАТОРНА РОБОТА №16

XQuery - мова запитів XML-документів

Виконав:

Яценко Владислав

ПМІ- 33

2025

Тема: Вивчення XQuery - мови запитів XML-документів

Мета роботи: Ознайомлення з синтаксисом мови XQuery та конструюванням запитів даних XML документа.

Хід роботи

Прочитав теоретичний матеріал.

XML документ з лабораторної роботи 11:


```xml
<insurance_database export_date="2025-10-20T17:52:58.325835+03:00" system="Insurance System v1.0">
    <agreements>
        <agreement id="5" status="Cancelled">
            <conclusion_date>2022-04-02</conclusion_date>
            <total_insurance_amount>430371.97</total_insurance_amount>
            <duration_months>24</duration_months>
            <insurance_policies>
                <policy id="680">
                    <start_date>2022-10-27</start_date>
                    <end_date>2023-10-22</end_date>
                    <insurance_amount>5019054.65</insurance_amount>
                    <insured_persons>
                        <insured id="80" person_ref="382">
                            <address>Одеса, вул. Петлюри, 29, кв. 20</address>
                            <phone_number>+380999289334</phone_number>
                            <email>client80@example.com</email>
                            <gender>M</gender>
                            <birth_date>1972-07-15</birth_date>
                            <insurance_objects>
                                <insurance_object id="317" type="Health">
                                    <insured_ref>80</insured_ref>
                                    <property_ref>234</property_ref>
                                    <property>
                                        <prop id="234" currency="UAH">
                                            <description>Nissan Qashqai, 2020 року випуску</description>
                                            <value>5638672.28</value>
                                            <year_of_issue>2008</year_of_issue>
                                            <location>Ірпінь, район ЖК "Затишний"</location>
                                        </prop>
                                    </property>
                                </insurance_object>
                            </insurance_objects>
                            <person>
                                <pers id="382">
                                    <first_name>Іван</first_name>
                                    <last_name>Кравченко</last_name>
                                    <gender>M</gender>
                                    <birth_date>1992-03-03</birth_date>
                                    <email>ivan.kravchenko@outlook.com</email>
                                    <phone_number>+380997386167</phone_number>
                                </pers>
                            </person>
                        </insured>
                    </insured_persons>
                </policy>
            </insurance_policies>
```

Запит 1: Всі активні договори

```
for $agreement in //agreement[@status='Active']
return
  <contract id="{$agreement/@id}">
    {$agreement/conclusion_date}
    {$agreement/total_insurance_amount}
    {$agreement/duration_months}
  </contract>
```

Result of the above expression applied to the above input file:

```
2023-10-16
1000853.76
36


2023-08-19
1888137.60
48


2024-04-25
2749832.93
24


2022-01-04
2461146.77
60


2024-10-15
3770385.01
48


2024-03-14
822036.22
36


2024-09-22
```

Запит 2: Договори з сумою більше середньої

```
let $avg := avg(//agreement/total_insurance_amount)
for $agreement in //agreement[total_insurance_amount > $avg]
order by $agreement/total_insurance_amount descending
return
  <high_value_contract>
    <id>{data($agreement/@id)}</id>
    <amount>{data($agreement/total_insurance_amount)}</amount>
    <status>{data($agreement/@status)}</status>
    <average>{$avg}</average>
  </high_value_contract>
```

Надіслати ☐ disable auto refresh ☐ disable syntax highlighting

Output Options: Node format:: text Output format:: adhoc ☐ Show types ☐ Hide variable names

Compatibility: Enable all extensions

Old languages: ○ XPath 2.0 ○ XPath 3.0 ○ XQuery 1.0 ○ XQuery 3.0

New languages: ○ XPath 4.0 ○ XQuery 4.0

Result of the above expression applied to the above input file:

```
25
4887697.36
Active
2.4066558915999997E6


85
4759705.89
Completed
2.4066558915999997E6


60
4684448.17
Active
2.4066558915999997E6


21
4400375.54
Active
2.4066558915999997E6


79
4394912.77
Completed
2.4066558915999997E6
```

Запит 3: Підрахунок за статусами

```
for $status in distinct-values(//agreement/@status)
let $count := count(//agreement[@status=$status])
where $count >= 1
order by $count descending
return
  <status_summary>
    <status>{$status}</status>
    <count>{$count}</count>
  </status_summary>
```

Надіслати ☐ disable auto refresh ☐ disable syntax highlighting

Output Options: Node format:: text Output format:: adhoc ☐ Show types ☐ Hide variable names

Compatibility: Enable all extensions

Old languages: ○ XPath 2.0 ○ XPath 3.0 ○ XQuery 1.0 ○ XQuery 3.0

New languages: ○ XPath 4.0 ○ XQuery 4.0

Result of the above expression applied to the above input file:

```
Active
35


Cancelled
6


Completed
6


Suspended
3
```

Запит 4: Особи з типом страхування

```
for $insured in //insured
let $type := $insured//insurance_object/@type
return
  <person>
    <name>{data($insured/person/pers/first_name)} {data($insured/person/pers/last_name)}</name>
    <phone>{data($insured/phone_number)}</phone>
    <insurance_type>
      {if ($type = 'Vehicle') then 'Транспорт'
       else if ($type = 'Health') then 'Здоровя'
       else if ($type = 'Life') then 'Життя'
       else if ($type = 'Property') then 'Майно'
       else 'Не вказано'}
    </insurance_type>
  </person>
```

Надіслати ☐ disable auto refresh ☐ disable syntax highlighting

Output Options: Node format:: text Output format:: adhoc ☐ Show types ☐ Hide variable names

Compatibility: Enable all extensions

Old languages: ○ XPath 2.0 ○ XPath 3.0 ○ XQuery 1.0 ○ XQuery 3.0

New languages: ○ XPath 4.0 ○ XQuery 4.0

Result of the above expression applied to the above input file:

```
Іван Кравченко
+380999289334

  Здоровя



Сергій Павленко
+380996546783

  Не вказано



Денис Левченко
+380991116190

  Здоровя



Сергій Франко
+380996394129

  Не вказано



Михайло Бондаренко
```

Запит 5: Договори з кількома застрахованими особами

```
for $agreement in //agreement
let $insured_count := count($agreement//insured)
where $insured_count > 1
order by $insured_count descending
return
  <multi_person_contract>
    <agreement_id>{data($agreement/@id)}</agreement_id>
    <status>{data($agreement/@status)}</status>
    <insured_persons_count>{$insured_count}</insured_persons_count>
    <total_amount>{data($agreement/total_insurance_amount)}</total_amount>
    <persons>
      {for $insured at $pos in $agreement//insured
       return
         <person number="{$pos}">
           <name>{data($insured/person/pers/first_name)} {data($insured/person/pers/last_name)}</name>
           <gender>{data($insured/gender)}</gender>
           <phone>{data($insured/phone_number)}</phone>
         </person>}
    </persons>
  </multi_person_contract>
```

Result of the above expression applied to the above input file:

```
15
Active
3
3770385.01


        Катерина Дмитренко
        M
        +380994369020

        Ніна Сидоренко
        M
        +380993072689

        Ніна Дмитренко
        F
        +380998921111




20
Active
3
822036.22


        Василь Олійник
        M
```

Запит 6: Статистика по валютах з умовою

```
for $currency in distinct-values(//prop/@currency)
let $properties := //prop[@currency=$currency]
let $total := sum($properties/value)
let $max_value := max($properties/value)
let $min_value := min($properties/value)
where count($properties) > 0
order by $total descending
return
  <currency_statistics>
    <currency>{$currency}</currency>
    <properties_count>{count($properties)}</properties_count>
    <total_value>{$total}</total_value>
    <average_value>{round($total div count($properties))}</average_value>
    <max_value>{$max_value}</max_value>
    <min_value>{$min_value}</min_value>
    <has_premium>
      {if (some $prop in $properties satisfies $prop/value > 5000000)
       then 'Так'
       else 'Ні'}
    </has_premium>
  </currency_statistics>
```

Result of the above expression applied to the above input file:

```
UAH
45
1.9501132148E8
4.333585E6
9.96905576E6
328801.11

  Так



EUR
13
7.315430536E7
5.627254E6
9.51763328E6
617415.62

  Так



USD
15
6.564778851E7
4.376519E6
8.33971362E6
730442.73

  Так
```

permalink, result-only

Контрольні запитання

1. Для чого застосовується мова XQuery?

XQuery (XML Query Language) застосовується для:

- Пошуку та вибірки даних з XML-документів
- Маніпулювання XML-даними - фільтрації, сортування, трансформації
- Створення зведених звітів з XML-структур
- Перетворення XML у інші формати (HTML, текст, JSON)
- Роботи з ієрархічними даними - запити до складних вкладених структур
- Об'єднання даних з декількох XML-документів

XQuery дозволяє працювати з XML-даними незалежно від способу їх зберігання, подібно до того, як SQL працює з реляційними базами даних.

2. Які особливості синтаксису мови XQuery?

Основні особливості синтаксису:

- Чутливість до регістру - name і Name - різні ідентифікатори
- Змінні починаються з $ - наприклад: $agreement, $total
- Коментарі в круглих дужках з двокрапкою - (: це коментар :)
- Рядки в лапках - одинарних 'text' або подвійних "text"
- Імена мають бути валідними XML-іменами - без спецсимволів
- Використовує XPath вирази для навігації по документу
- Оператор кома для об'єднання послідовностей
- Два типи порівняння:
  - Загальні: =, !=, <, >, <=, >=
  - Величин: eq, ne, lt, gt, le, ge

3. Що означає вираз FLWOR?

FLWOR - це абревіатура від п'яти ключових слів:

- F - for - визначає змінні циклу, ітерація по елементах
- L - let - оголошує змінні, присвоює значення
- W - where - задає умови фільтрації
- O - order by - сортує результати
- R - return - визначає що повертати

```
for $змінна in вираз
let $інша_змінна := значення
where умова
order by поле
return результат
```

4. Наведіть приклад використання виразу QuantifiedExpr

QuantifiedExpr - це квантори існування:

- some - хоча б один елемент задовольняє умову (∃)
- every - всі елементи задовольняють умову (∀)

Приклад 1: some (існує)

```
for $agreement in //agreement
where some $insured in $agreement//insured satisfies $insured/gender = 'F'
return
  <female_client_contract>
    {$agreement/@id}
    {$agreement/@status}
  </female_client_contract>
```

Приклад 2: every (для всіх)

```
for $agreement in //agreement
where every $insured in $agreement//insured satisfies $insured/gender = 'M'
return
  <male_only_contract>
    {$agreement/@id}
  </male_only_contract>
```

5. Наведіть приклад використання виразу SwitchExpr

SwitchExpr - це аналог switch/case для вибору значення за умовою.

Приклад 1: Переклад статусу

```
for $agreement in //agreement
return
  <contract>
    {$agreement/@id}
    <status_ukrainian>
      {switch ($agreement/@status)
       case 'Active' return 'Активний'
       case 'Cancelled' return 'Скасований'
       case 'Completed' return 'Завершений'
       case 'Suspended' return 'Призупинений'
       default return 'Невідомий статус'}
    </status_ukrainian>
  </contract>
```

Приклад 2: Категорії за сумою

```
for $agreement in //agreement
let $amount := $agreement/total_insurance_amount
return
  <contract>
    {$agreement/@id}
    <category>
      {switch (true())
       case $amount > 5000000 return 'VIP'
       case $amount > 3000000 return 'Premium'
       case $amount > 1000000 return 'Standard'
       default return 'Basic'}
    </category>
  </contract>
```

6. Як використовується вираз TryCatchExpr?

TryCatchExpr використовується для обробки помилок під час виконання запиту.

Синтаксис:

try { *код, який може викликати помилку* }

catch * { *код обробки помилки* }

Приклад: Безпечне обчислення середнього

```
try {
  let $avg := avg(//agreement/total_insurance_amount)
  return
    <result>
      <average>{$avg}</average>
      <status>Success</status>
    </result>
}
catch * {
  <result>
    <average>0</average>
    <status>Error calculating average</status>
  </result>
}
```

Коли використовувати:

- При роботі з необов'язковими даними
- При математичних операціях (ділення, корінь)
- При конвертації типів даних
- При роботі з зовнішніми джерелами даних
-

Висновок: на цій лабораторній роботі я практично ознайомився з синтаксисом мови XQuery та конструюванням запитів даних XML документа.